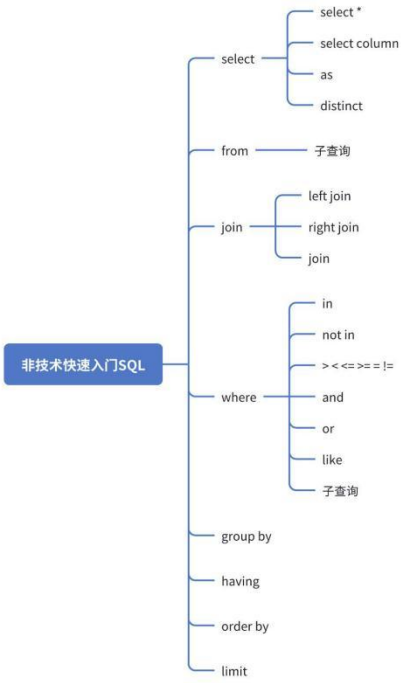
SQL-牛客网刷题（快速入门部分）

一、SQL 常用关键字



二、非技术快速入门刷题

1.查询所有列 //select * from table

```
SQL
select * from user_profile
```

2.查询多列 // select columns

```
SQL
select
device_id
,gender
,age
,university
```

```
from user_profile
```

3.查询结果去重 // distinct

```
SQL
select
university
from user_profile
group by 1
```

4.查询结果限制返回行数 // limit

```
SQL
select
device_id
from user_profile
limit 2
```

5.将查询后的列重新命名 // as 重命名

```
SQL
select
device_id as user_infos_example
from user_profile
limit 2
```

6.查找学校是北大的学生信息 // where 查询

```
SQL
select
device_id
,university
from user_profile
where university = '北京大学'
```

7.查找年龄大于 24 岁的用户信息 // where 查询

```
SQL
```

```
select
device_id
,gender
,age
,university
from user_profile
where age > 24
```

8.查找某个年龄段的用户信息 // where and

```
SQL
select
device_id
,gender
,age
from user_profile
where age >= 20 and age <= 23
```

9.查找除复旦大学的用户信息 // where !=

```
SQL
select
device_id
,gender
,age
,university
from user_profile
where university != '复旦大学'
```

10.用 where 过滤空值练习 // where is not null

```
SQL
select
device_id
,gender
,age
,university
from user_profile
where age is not null
```

11.高级操作符练习(1) // where and

```SQL
select
device_id
,gender
,age
,university
,gpa
from user_profile
where gpa > 3.5 and gender = 'male'
```

12.高级操作符练习(2）// where or

```SQL
select
device_id
,gender
,age
,university
,gpa
from user_profile
where university = '北京大学' or gpa > 3.7
```

13.Where in 和 Not in // where in

```SQL
select
device_id
,gender
,age
,university
,gpa
from user_profile
where university in ('北京大学', '复旦大学', '山东大学')
```

14.操作符混合运用 // where and or

```SQL
select
```

```
device_id
,gender
,age
,university
,gpa
from user_profile
where (gpa > 3.5 and university = '山东大学')
or (gpa > 3.8 and university = '复旦大学')
```

15.查看学校名称中含北京的用户 // where like

```
SQL
select
device_id
,age
,university
from user_profile
where university like '%北京%'
```

16.查找 GPA 最高值 // max

```
SQL
select
round(max(gpa), 1) as max_gpa
from user_profile
where university = '复旦大学'
```

什么时候用 round(数值, N)

要求指定小数位数、四舍五入时使用。本题要求保留 1 位小数，必须写

17.计算男生人数以及平均 GPA // avg

```
SQL
select
count(1) as male_num
,avg(gpa) as avg_gpa
from user_profile
where gender = 'male'
```

COUNT 本质：统计行数

COUNT(*)：

统计所有行，NULL 不影响，全部计数。

COUNT(常量)

例：COUNT(1) 每行填充固定值（永远非空），全部计数，不会漏掉 NULL 行。

写 1 只是习惯，写 2、'a' 效果一样。

统计总人数 / 总行数首选。

COUNT(列名)

例：COUNT(gpa) 只统计该列不为 NULL 的行；

这一列 NULL，改行直接忽略

18.分组计算练习题 // count、avg、group by

```SQL
select
gender
,university
,count(device_id) as user_num
,round(avg(active_days_within_30), 1) as avg_active_day
,round(avg(question_cnt), 1) as avg_question_cnt
from user_profile
group by 1, 2
order by gender,university
```

1.为什么要用 group by

2.为什么要用 order by

GROUP BY 用来分类计算

ORDER BY 用来整理顺序

1. GROUP BY 分组

作用：把数据归类分组，配合聚合函数，每组单独计算

本题：group by gender,university

将【同一学校+同一性别】的用户归为一组，分别统计人数、平均值

比如这道题，先把男性在北京大学的分成一组，然后计算

在把男性在清华大学的分成一组，计算

男性计算完了，把女性在北京大学的分为一组，然后计算

在把女性在清华大学的分成一组，计算

总之就是，gruop by 在把数据按照后面的要求先分组

最后结果，按你指定的字段切分成很多小分组，每组输出 1 行

重点规则：

SELECT 只能写：分组字段+count/avg/sum 等聚合函数

没有 GROUP BY：整张表当成 1 组，只输出 1 行汇总数据

2. ORDER BY 排序

作用：对最终查询结果调整展示先后顺序

GROUP BY 仅仅是归类，不会自动排序。

本题要求：按照 gender、university 升序展示，必须手动添加。

order by gender,university

这句话意思是先按 gender 升序，gender 相同，再按 university 升序。

19.分组过滤练习题 // avg、group by、having 过滤

```SQL
select
university
,round(avg(question_cnt), 3) as avg_question_cnt
,round(avg(answer_cnt), 3) as avg_answer_cnt
from user_profile
group by 1
having avg(question_cnt) < 5 or avg(answer_cnt) < 20
```

1.什么时候用 having

WHERE：原始行过滤（分组之前筛选数据）

HAVING：分组之后过滤组（GROUP BY 之后，筛选聚合计算出来的结果）

2.为什么 having 后面还要用 avg 计算，而不是用 avg_question_cnt 这些

SQL 书写顺序：SELECT → FROM → WHERE

执行顺序：FROM → WHERE → GROUP BY → 聚合计算 → HAVING → SELECT

WHERE 不能放聚合函数的根本原因：

执行阶段，还没进行分组、还没有算出 avg、count 这类结果

3.子查询方案

```
select
*
from
(

    select
    university
   ,round(avg(question_cnt),3) as avg_question_cnt
   ,round(avg(answer_cnt),3) as avg_answer_cnt
   from user_profile
   group by 1
) as a
where avg_question_cnt<5 or avg_answer_cnt<20
```

核心思路一句话

内层子查询：先执行分组聚合，把【每个学校+平均发帖、平均回帖】计算完毕，生成一张**临时虚拟表 a**

虚拟表 a 里面：avg_question_cnt **不再是聚合函数！它变成了普通字段**

外层 WHERE：筛选虚拟表里的普通字段，自然合法，不会报错

4.对比子查询和 having 写法

HAVING 方案（推荐本题使用）
数据库在分组后直接过滤，不需要生成完整临时表，性能更好，代码简短
子查询方案（适合复杂场景）
① 筛选条件复杂，HAVING 可读性很差
② 需要多次利用聚合结果做多层筛选

20.分组排序练习题 // order by

```SQL
select
university
,avg(question_cnt) as avg_question_cnt
from user_profile
group by 1
order by 2
```

21.浙江大学用户题目回答情况 //left join 表连接 + where 筛选

```SQL
select
a.device_id as device_id
,b.question_id as question_id
,b.result as result
from user_profile a
left join question_practice_detail b
on a.device_id = b.device_id
where a.university = '浙江大学'
order by 2
```

1.为什么两个文件要起名为 a，b？

（1）为了简写
question_practice_detail 原名太长，每次写完整名字麻烦
语法：表名 别名
原写法：question_practice_detail.device_id
简写：a.device_id
规则：空格直接命名，不需要 as；

（2）为了区分
多张表有同名字段的时候必须用别名区分
两张表都有 device_id 你只写 device_id，数据库不知道你要哪一张表的，会报错！
所以必须 a.device_id / b.device_id

2.ON 的语法是什么？什么时候用 ON？

（1）语法格式
表 1 连接类型 join 表 2
on 匹配条件
（2）ON 作用
专门规定：两张表依靠哪一列进行匹配拼接

3.a.device_id as device_id

（1）as = 别名关键字
可以省略 a.device_id device_id 等价，效果一模一样
（2）为什么 a.device_id、question_id、result 可以省略起名这个步骤？
question_id 原本字段名就叫 question_id 不需要改名，直接输出，表头自动沿用原名

22.统计每个学校的答过题的用户的平均答题数 //left join 表连接 + 聚合

```SQL
select
b.university as university
,round(count(1) / count(distinct a.device_id), 4) as
avg_answer_cnt
from question_practice_detail a
left join user_profile b
on a.device_id = b.device_id
group by 1
order by 1
```

1、为什么要 from question_practice_detail，而不是 user_profile

方案 1 ☑ FROM 答题表 a（你的写法）
数据源起点：所有答题记录
每条答题记录，关联找到这个人属于哪所学校。
天然只保留【有答题记录的用户】，完美匹配题意：只统计答过题的用户
方案 2 ❌ 如果 FROM 用户表 user_profile
起点是全部用户（包含从来没有答过题的人）
一旦 left join 答题表，没答题的用户会匹配不到答题记录，后续统计容易出错，还要额外过滤，代码变复杂。
一句话总结选择基准（记下来刷题通用）
你要统计的事件（答题、订单、浏览）在哪张表，就把这张表放在 FROM（左表）！
答题行为在 question_practice_detail→FROM 这张表

23.统计每个学校各难度的用户平均刷题数 //left join 表连接 + 聚合

```SQL
select
b.university,
c.difficult_level,
round(count(1) / count(distinct a.device_id),4) as avg_answer_cnt
from question_practice_detail a
left join user_profile b
on a.device_id = b.device_id
left join question_detail c
on a.question_id = c.question_id
group by b.university, c.difficult_level
```

```
order by b.university, c.difficult_level;
```

1、SELECT 输出 3 列，group by 只写 2 列？（初学者最大卡点）

SELECT 中非聚合字段，必须全部出现在 GROUP BY 后面！

group by 学校,难度：
数据库把数据打包成一组一组：{北京大学,hard} 一组、{复旦大学,easy} 一组，每一组最终只输出一行
第三列 avg_answer_cnt，是针对这一组内部数据计算出来的值
它是分组之后的结果，不是分组依据，自然不需要写在 group by

2、为什么 count(distinct a.device_id)而不是 b.device_id?

（1）所有基础数据源头来自 a（答题记录），a.device_id 一定存在，不会为 NULL
（2）假设出现一条脏数据：
有答题记录 a.device_id = 9999，但是 user_profile 里面不存在这个 device_id。
拼接之后这一行数据：
a.device_id = 9999
b.device_id = NULL
university = NULL
👉count (distinct b.device_id)
👉读取 b.device_id，值是 NULL
👉 count 函数会直接忽略 NULL，这条答题记录对应的用户直接消失，统计人数变少，计算出错 ❌

3、为什么要写 order by

没有 order by，查询结果顺序随机
判题系统，会逐行比对结果 如果数据库输出顺序和样例顺序不一致，就算数据全部正确，系统直接判定答案错误

24.统计每个用户的平均刷题数//left join 表连接 + where 筛选

```SQL
select
b.university as university
,c.difficult_level as difficult_level
,count(1) / count(distinct b.device_id) as avg_answer_cnt
from question_practice_detail a
left join user_profile b
on a.device_id = b.device_id
and b.university = '山东大学'
left join question_detail c
on a.question_id = c.question_id
where b.device_id is not null
group by 1, 2
```

1、为什么要判断不为空 where b.device_id is not null

因为 left join user_profile b 的逻辑：
只有同时满足【设备相同 + 山大用户】，才能成功匹配 b 表数据；
其他所有用户：匹配失败 → b 表全部字段 = NULL
判断不为空的作用：剔除匹配失败的数据，只保留山大用户
如果不写这个，则最后结果为：
全部用户的答题记录
山大用户带上学校信息；其他学校用户 b 表字段全部 NULL

2、联系 23 题的解法，只用多写一个 where

```sql
select
    b.university,
    c.difficult_level,
    round(count(1) / count(distinct a.device_id),4) as avg_answer_cnt
from question_practice_detail a
left join user_profile b
on a.device_id = b.device_id
left join question_detail c
on a.question_id = c.question_id
where b.university = '山东大学'
group by b.university, c.difficult_level
order by b.university, c.difficult_level;
```

3、为什么用 join 改写后不用判断是否为空（join = inner join）

left join user_profile b--->**join user_profile b**
内连接机制：
只保留两边匹配成功的数据
只要匹配不成功，整行直接丢弃，不会产生任何 NULL 数据

4.left join 和 join 区别

核心本质
INNER JOIN(join)：只保留两边匹配成功的数据，匹配不上直接丢弃整行
LEFT JOIN：左表全部保留；右表匹配不到，右表字段全部填充 NULL

应用场景（什么时候用哪个，重点）
✓ INNER JOIN（普通 join）
场景：只想要两边都有关联成功的数据，不需要匹配失败的数据
只要关联不上，这行我就不要
例子：
查询有答题记录的用户信息，用户表 join 答题表；没有答题的用户直接过滤掉
查询用户+他的学校信息，只保留学校存在的用户
特点：结果行数一定不会大于左表行数；不会产生 NULL

✓ LEFT JOIN（左连接）
场景：左表所有记录都要保留，哪怕右表找不到对应数据也不能删掉左表行
即使右表没有匹配，左表数据不能丢，右表字段填 null
例子：
查询全部用户，附带他们的订单；没有订单的用户也要展示出来，订单字段为 null
用户表 left join 答题表：所有用户都输出，没答题的用户答题字段 null

25.查找山东大学或者性别为男生的信息 // union all

```SQL
select
device_id
,gender
,age
,gpa
from user_profile
where university = '山东大学'

union all

select
device_id
,gender
,age
,gpa
from user_profile
where gender = 'male'
```

1、为什么不能用 or

```
select
device_id,gender,age,gpa
from user_profile
where university='山东大学' or gender='male'
```

读题重点（两个硬性要求）

1. 条件：学校为山东大学 **OR** 性别为男性，**结果不去重**
2. 输出顺序：**先输出山东大学的全部用户，再输出全部男性用户**

看样例输出：

第一部分：山东大学 2 条（5432、2131）

第二部分：全部男性一共 5 条（2138，3214，5432，2131，4321）

👉 5432、2131 既是山东大学，又是男性。 **所以最终结果这两条会出现 2 次，不去重！**

where university='山东大学' or gender='male' 做了什么？

or 是**条件筛选**，只要满足任意一条就保留这一行。

只会得到：满足山大 或者 是男生 的**不重复的行**。

5432、2131 只会出现**一次**，不会重复出现两次。

但是题目要求： 先把山大全部拿出来，再把全部男性拿出来，**两部分直接拼接，允许重复**。

or 不会把两个集合分别查询再拼接；它是对原始表每一行判断条件，一行最多输出一次

✅正确语法：UNION ALL
union all：直接把多个 select 结果上下拼接，不去重（本题必须用）
union：拼接之后自动去重，本题不能用

26.计算 25 岁以上和以下的用户数量 // case when

```SQL
select
case when age < 25 or age is null then '25岁以下'
     when age >= 25 then '25岁及以上'
end as age_cut
,count(1) as number
from user_profile
group by 1
```

CASE WHEN##

✅什么时候用 case when

>需要对每一行做「条件判断，生成新字段」的时候用

3 类高频使用场景：

1. 做数值区间划分：年龄分段、分数分级（本题场景）
2. 值翻译：male→男，female→女
3. 行内多条件分支计算：不同条件输出不同文字/数字

对比：

> where：用来过滤删掉行

> case when：不删除行，给每一行算出一个新列

两种语法（只记第一种，95%刷题够用）

①搜索 case（推荐，支持> < or and）

```
CASE
    WHEN 条件1 THEN 返回值1
    WHEN 条件2 THEN 返回值2
     ELSE 默认值
END AS 新列名
```

规则：

1. 从上往下判断，**命中第一个条件就结束**
2. 都不满足，走 ELSE；没有 ELSE 返回 NULL

示例：年龄分段

```
case
    when age<25 or age is null then '25 岁以下'
    when age>=25 then '25 岁及以上'
end as age_cut
```

②简单 case，只做等于匹配，很少用

```
CASE gender
     WHEN 'male' THEN '男'
     WHEN 'female' THEN '女'
END AS sex_cn
```

常用搭配

1. 搭配 GROUP BY

case 生成分类列，直接按这个分类做统计。

⚠3 个必踩坑

1. null <25 不成立！null 不能用><判断，必须写 age is null
2. 区间注意边界，>25 和>=25 不一样，漏边界会产出 NULL 分组
3. 条件顺序不能乱，范围大的条件写在后面

快速判断用不用 case 小测试

1. 我要删掉一部分行 → 用 where
2. 我不想删行，给每行打标签、分段、翻译值 → 用 case when

27.查看不同年龄段的用户明细// case when

SQL

```
select
device_id
,gender
,case when age < 20 then '20岁以下'
     when age >= 20 and age <= 24 then '20-24岁'
     when age >= 25 then '25岁及以上'
 else '其他' end as age_cut
from user_profile
```

```sql
select
device_id,
gender,
case
    when age < 20 then '20岁以下'
    when age >= 20 and age <= 24 then '20-24岁'
    when age >= 25 then '25岁及以上'
else '其他' end as age_cut
from user_profile;
```

1、为什么不要 group by

看题目原文

分别查看不同年龄段用户的**明细情况**，取出相应数据。

明细 = 每一个用户单独输出一行

原始表一共 7 条用户记录，输出结果也必须是 7

① 如果不加 group by（正确做法）

from user_profile 读出 7 行原始数据

case when 给**每一行单独计算一个 age_cut 标签**

每一行保留原样，只是多出来一个新标签字段。

输入 7 行 →输出 7 行。

② 如果写上 group by device_id,gender

group by 会把相同 device_id+gender 的行合并为 1 行

本题 device_id 全部不重复，所以结果行数碰巧还是 7 行，**不会报错，但是完全没必要**

group by 存在的意义

是配合聚合函数 count / sum / avg，做统计汇总

什么时候才该写 group by?

例：统计每个年龄段有多少人

通俗大白话记忆

明细查询：看每个人本身的数据 →不要 group by

我要看张三、李四、王五各自信息，一人一行
case 只是给每个人贴个标签，人还是独立的

分组统计：看归类之后的总和，不关心个体 →要 group by
我不关心张三李四分别是谁
我只想知道：20- 24 岁一共多少人，25 岁以上一共多少人
把一堆人压缩成一行数字

本题回顾
题目输出字段包含 device_id、gender，是**每个用户的个人信息**，属于明细

任务只是给每行打年龄段标签，**不需要合并、不需要统计 count/sum**，因此完全不需要 group by

28.计算用户 8 月每天的练题数量 // left 函数

```SQL
select
day(`date`) as day
,count(1) as question_cnt
from question_practice_detail
where left(`date`, 7) = '2021-08'
group by 1
```

1、date 为什么要么都不加反引号，要么都加反引号（注意：不是单引号）

写法	含义	概念
`date`	引用表头名字	**列名**
`` `date` ``	反引号包裹表头名字，规避关键字冲突	**列名**
`'2021-08-13'`	一个具体的日期文本数据	**字段值（常量）**
`'date'`	普通字符串文本 `date`，不是列	**字段值（常量）**

2、day 函数

作用：提取日期的「日」部分，返回几号（数字）

```
day('2021-08-13') → 13
day('2021-08-18') → 18
```

参数：日期类型的字段/日期字符串，不要给字符串加引号把列名包起来

配套同类日期函数：

month(date)：取月份

year(date)：取年份

date(date)：把 datetime 转为日期 yyyy-mm-dd

3、left 函数- 字符串截取函数

left(字符串,截取长度 n)

作用：从字符串左边开始，截取 n 个字符。

示例： date='2021-08-13' left(date,7) →截取前 7 位 →'2021-08'

4、补充：date_format(日期字段, '格式模板')

date_format(date,'%Y-%m')

原始日期：2021-08-13

格式化输出字符串：'2021-08'

所以 date_format(date,'%Y-%m') = '2021-08'

含义：筛选 2021 年 8 月的全部数据。和 left(date,7)='2021-08'效果等价

👉优先推荐 date_format：真正日期类型字段也可以正常工作；left 函数依赖把日期转字符串。

```
select
day(date) as day,
count(1) as question_cnt
from question_practice_detail
where date_format(date,'%Y-%m') = '2021-08'
group by 1;
```

29. 计算用户的平均次日留存率 // date_add 函数、with 临时表

```
SQL
-- 用户注册表
-- 用户活跃表
```

```
with tmp
as(
    select
    device_id
    ,date
    from question_practice_detail
    group by 1, 2
)

select
count(b.date) / count(a.date)
from tmp a
left join tmp b
on a.device_id = b.device_id
and a.date = date_sub(b.date, interval 1 day)
```

1、两种口径

口径 1：每日留存率求平均

第一天算一个留存率：0.5

第二天算一个留存率：0

把这两个数字拿来求平均：（0.5 + 0）÷ 2 = 0.25

👉含义：**把每一天当成同等重要，一天只贡献一个数字。不管这天来了 2 个人还是 100 个人，两天权重一样。**

```
select count(t2.device_id)/count(t1.device_id) as avg_ret
from (
    select distinct device_id, date from question_practice_detail
) t1
left join (
    select distinct device_id, date from question_practice_detail
) t2
on t1.device_id = t2.device_id
and DATEDIFF(t2.date, t1.date) = 1;
```

口径 2：全部用户- 天摊在一起直接相除

我们先统计全部的用户- 天记录：

8- 01：2 个活跃用户，其中 1 个人次日留存成功

8- 02：100 个活跃用户，0 个人次日留存成功

总活跃用户=2+100=102

总留存成功用户=1+0=1

整体比值 = 1 ÷ 102 ≈ 0.0098

👉含义：**每一个用户都算一份权重。哪天用户多，哪天对结果影响就巨大**

```
with tmp
as(
    select
    device_id
```

```
    ,date
    from question_practice_detail
    group by 1, 2
)
select
count(b.date) / count(a.date) as avg_ret
from tmp a
left join tmp b
on a.device_id = b.device_id
and b.date = date_add(a.date, interval 1 day);
```

对比上面案例的结果
口径 1（每日求率，再平均）：0.25
口径 2（全部用户摊平直接除）：≈0.0098
两套都是“平均留存”，但是数字完全不一样。根源就是：权重给谁。
1：权重给**日期**，一天 1 票。
2：权重给**用户个体**，一个用户 1 票

2、这道题用的口径 2

```
count(b.date) / count(a.date) as avg_ret
```

这一句，相当于就是 `Σ留存用户数 / Σ活跃用户数`
为什么要用 count 呢？
count(列)：**不会统计该列为 null 的行。**
b.date 为 null ➡用户次日没有刷题，不计入分子。
count (a.date)：统计全部基准活跃用户，作为分母。

3、with ... as

作用：把一段查询结果存成临时虚拟表 tmp，后面可以重复使用。
场景：需要多次复用同一套子查询结果，替代多层嵌套子查询，可读性更高。

4、自连接

```
from tmp a
left join tmp b
```

一张表起两个别名 a、b，逻辑当做两张表
a：基准，用户某天发生刷题行为。
b：用来查找该用户的次日刷题记录

5、date_add(日期, interval 1 day) 日期函数

```
 b.date = date_add(a.date, interval 1 day)
```

含义：b 的日期 = a 日期的后一天，也就是次日
等价写法：
（1）`a.date = date_sub(b.date, interval 1 day)`

date_sub 和 date_add 相反，表示前一天
这句的含义：b 日期的前一天等于 a 日期
（2）`datediff(b.date,a.date)=1`
datediff 函数：表示这两个日期相差多少，这里=1，就 b 和 a（b>a）差 1

6、为什么口径 1 要用 group 而口径 2 不用

场景：统计考试

表里面是每个学生的分数，每个学生一行。
方式 1：group by 班级

```
select avg(score) from table group by class;
```

按班级分组，输出每一个班的平均分。得到一堆班级平均分，之后可以外层再求平均
相当于口径 1：group by date，输出每一天的留存率
分组 = 拆成小组，每组算一个指标

方式 2：不写 group by

```
select avg(score) from table;
```

不分组，全部学生当成 1 个大组，直接算全体学生的平均分
相当于口径 2：不 group by，全部用户一天直接算总留存率。
不分组=全部数据当成一个大组，只输出 1 个结果。

一句话记忆
group by：**切分小组，每组算一个结果，输出多行**
无 group by：**全部数据作为 1 个大组，聚合只输出 1 行**

30.统计每种性别的人数 // substring_index 函数

```
SQL
select
substring_index(profile,',',-1) as gender
,count(device_id) as number
from user_submit
group by 1
```

1、substring_index(字符串,分隔符,计数)
作用：按指定分隔符切割字符串，取出**第 N 次分割位置之前 / 之后的内容**
语法：`substring_index(str, delim, count)`

```
str：要切割的原始字符串
delim：分隔符号，本题是逗号 ，
count：
正数 n：从左边数，遇到第 n 个分隔符，取左边全部内容
负数- n：从右边数，遇到第 n 个分隔符，取右边全部内容
```

31.提取博客 URL 中的用户名 // substring_index 函数

```
SQL
select
device_id
,substring_index(blog_url, '/', -1)
from user_submit
```

1、到底什么时候要用 group，什么时候不用

✓需要 group by 的场景

（1）需求动词出现：统计、计算、求和、计数、求平均、分组对比

（2）输入很多行，输出行数变少（多行合并成一组）

（3）select 里面出现聚合函数：count() sum() max() min() avg()

32.截取出年龄 // substring_index 函数

```
SQL
select
substring_index(substring_index(profile, ',', -2), ',', 1) as age
,count(1) as number
from user_submit
group by 1
```

1、`substring_index(substring_index(profile,',',-2),',',1) as age`

一次 substring_index 只能拿从左数前 N 段，或者从右数后 N 段，拿中间片段必须嵌套。

33.找出每个学校 GPA 最低的同学 // 聚合+left join+from

子查询

```SQL
select
t2.device_id as device_id
,t1.*
from
(
    select
    university
    ,min(gpa) as gpa
    from user_profile
    group by 1
) t1
left join
(
    select
    university
    ,device_id
    ,gpa
    from user_profile
) t2
on t1.university = t2.university
and t1.gpa = t2.gpa
order by 2
```

1、join 写法

```
select b.device_id,b.university,b.gpa
from user_profile b
join(
    select
    university
    ,min(gpa) as min_gpa
    from user_profile
    group by university
) a
on b.university = a.university
and b.gpa = a.min_gpa
order by b.university asc
```

2、子查询

（1）SQL 子查询（派生表）核心知识点

当 select 写在 from ()里面，这叫派生表（子查询作为临时表）

（1）子查询和外部主 SQL 的关系：

子查询先运行，产出中间结果；

外层主查询把它当成一张普通表，做 join、select、where。

3、什么时候用子查询

触发你使用【from (子查询) t】的触发条件（做题脑子判断）

✅满足下面全部，就用子查询派生表：

需要分组：group by A（学校）

需要求最值：min () /max ()

但是！我还想要拿到这条最值对应的原始行的其他字段（本题就是 device_id）

简单大白话：

group by 之后，我只得到数字，但是我想要这个数字背后对应的完整一行数据 → 必须子查询 + join

4、总结思路

- 需求：每个学校最低 gpa，**同时要 device_id**
- group by university 可以拿到学校 + min (gpa)，但是拿不到 device_id
- 把分组聚合丢进子查询 a，先算出每个学校最低 gpa
- b 就是原始整张表，保留 device_id
- join 匹配：学校一样，gpa 等于该校最低 gpa，把 device_id 拼回来

34.统计复旦用户 8 月练题情况 // union all

```SQL
select
a.device_id as device_id
,b.university as university
,count(1) as question_cnt
,sum(if(result = 'right', 1, 0)) as right_question_cnt
from question_practice_detail a
left join user_profile b
on a.device_id = b.device_id
and left(a.date , 7) = '2021-08'
and b.university = '复旦大学'
where b.device_id is not null
```

```
group by 1, 2

union all
select
a.device_id as device_id
,a.university as university
,0 as question_cnt
,0 as right_question_cnt
from user_profile a
left join question_practice_detail b
on a.device_id = b.device_id
and a.university = '复旦大学'
where b.device_id is null
and a.university = '复旦大学'
```

1、参考这个

```
select
    up.device_id,up.university,
    count(qpd.question_id) as question_cnt,
    sum(if(qpd.result='right',1,0)) as right_question_cnt
from user_profile up
left join question_practice_detail qpd
on up.device_id=qpd.device_id
and date_format(qpd.date,'%Y-%m') = '2021-08'
where up.university='复旦大学'
group by up.device_id,up.university
```

2、思路解析

要保留【复旦全部用户】，哪怕没有做题记录 →必须把复旦用户表作为左表，使用 LEFT JOIN

左表：复旦大学所有用户（来自 user_profile）

右表：8 月份的做题记录（question_practice_detail）

第一步：先拿到所有复旦用户，作为左表

从 user_profile 筛选 university='复旦大学'，得到全部复旦学生：3214、4321。

这一步保证：不管做没做题，两个用户一定都在结果里。

第二步：准备右表：只提取 8 月份做题记录

⚠重要：时间条件写在 join 的 on 后面，不要写 where！

如果写 where，left join 会降级变成 inner join，没做题的用户直接被删掉。

第三步 left join 关联两张表

左表（复旦全部用户） LEFT JOIN 右表（8 月做题记录）

用户 3214：8 月有做题 →匹配到多条做题记录

用户 4321：8 月没有做题 →右表全部字段为 null

第四步：分组聚合统计每个用户

count(右表.question_id)：统计做题总数。

count (字段) 遇到 null 不计入，8 月无做题自动得到 0。

sum(if(result='right',1,0))统计答对数量；没有做题全部 null，sum 得到 0【if 判断：答对记 1，答错记 0；sum 求和，得到答对总题数】

第五步：按 device_id，university 分组

35.浙大不同难度题目的正确率 // sumif

```SQL
select
c.difficult_level as difficult_level
,sum(if(a.result = 'right', 1, 0)) / count(1) as correct_rate
from question_practice_detail a
left join user_profile b
on a.device_id = b.device_id
and b.university = '浙江大学'
left join question_detail c
on a.question_id = c.question_id
where b.device_id is not null
group by 1
order by 2
```

36. 查找后排序 // order by

```SQL
select
device_id
,age
from user_profile
order by 2
```

37.查找后多列排序 // order by

```SQL
select
device_id
,gpa
,age
from user_profile
order by 2, 3
```

38.查找后降序排列 // order by desc

```SQL
select
device_id
,gpa
,age
from user_profile
order by 2 desc, 3 desc
```

39.21 年 8 月份练题总数 // count(distinct)

```SQL
select
count(distinct device_id) as did_cnt
,count(1) as question_cnt
from question_practice_detail
where left(`date`, 7) = '2021-08'
```